# Esencial Examen Estructuras de Control y Bucles en Java - José Ángel Alejo Sillero

Las **estructuras de control en Java se dividen en Alternativas y Bucles**.

En las **Alternativas** tenemos: Simples (IF), los Dobles (IF, ELSE) y Múltiples (switch normal, switch moderno y if-else anidado).

En los **Bucles** tenemos: While, Do_While y For.

1. **Simples - IF** → Se utilizan para establecer qué pasará si se cumple una condición (IF - si)

```
if (condición) {
    // Código a ejecutar si la condición es verdadera
}
```

2. **Dobles - IF, ELSE** → Se utilizan para establecer qué pasará si se cumple una condición (IF - si) y qué pasará si no se cumple (ELSE - si no)

```
if (condición) {
    // Código a ejecutar si la condición es verdadera
} else {
    // Código a ejecutar si la condición es falsa
}
```

3. **Múltiples**

    1. **Switch** → Se utilizan para seleccionar una de entre varias opciones basadas en el valor de una variable.

        1. **Switch "Normal"**

```
switch (variable) {
    case valor1:
        // Código a ejecutar si variable == valor1
        break;
    case valor2:
        // Código a ejecutar si variable == valor2
        break;
    // Más casos...
    default:
        // Código a ejecutar si variable no coincide con ningún caso
}
```

        2. **Switch "Moderno"**

```
switch (variable) {
    case valor1 -> {
        // Código a ejecutar si variable == valor1
    }
    case valor2 -> {
        // Código a ejecutar si variable == valor2
    }
    // Más casos...
    default -> {
        // Código a ejecutar si variable no coincide con ningún caso
    }
}
```

2. **IF-ELSE ANIDADOS** → Se utilizan para evaluar múltiples condiciones de manera secuencial.

```
if (condición1) {
    // Código a ejecutar si condición1 es verdadera
} else if (condición2) {
    // Código a ejecutar si condición2 es verdadera
} else {
    // Código a ejecutar si ninguna de las condiciones anteriores es
verdadera
}
```

4. **BUCLES → While | Do While | For**

1. **While** → Se utiliza cuando NO sabemos cuántas veces se debe repetir un bloque de código.
   Se lee como "mientras".

```
while (condición) {
    // Código a ejecutar mientras la condición sea verdadera
}
```

2. **Do-While** → Igual que el while, pero el bloque de código se repite al menos una vez ANTES de evaluar la condición.
   Se lee como "haz... mientras".

```
do {
    // Código a ejecutar al menos una vez y luego mientras la condición sea
verdadera
} while (condición);
```

3. **For** → Se utiliza cuando SÍ sabemos cuántas veces se debe repetir un bloque de código.
   Se lee como "para" inicialización,(hasta que) condición, incremento/decremento (de la inicialización).
   Se puede incrementar o decrementar. Si es incremento se usa "++", si es decremento se usa "--". Si es

otro valor se usa "+=" o "-=" y el valor a incrementar o decrementar. Por ejemplo, para incrementar de 2 en 2 se usa "i +=2".

```
for (inicialización; condición; incremento/decremento) {
    // Código a ejecutar en cada iteración
}
```

---

# Tabla comparativa de estructuras de control y bucles en Java

| Estructura | Cuándo se utiliza | Sintaxis Básica | Se lee |
|---|---|---|---|
| **If** | Evaluar una condición y ejecutar código si es verdadera | if (condición) { // código } | Si (condicion) se cumple, ejecutar código. |
| **If-Else** | Evaluar una condición y ejecutar código si es verdadera o falsa. | if (condición) { // código } else { // código } | Si (condicion) se cumple, ejecutar código. Si no, ejecutar otro código. |
| **Switch** | Seleccionar entre múltiples opciones basadas en el valor de una variable | switch (variable) { case valor1: // código; break; ... } | Si (variable) es igual a valor, ejecutar código asociado. |
| **BUCLES** | **Repetir un bloque de código mientras se cumpla una condición**. | | |
| **While** | Cuando **no sabemos cuántas veces** se debe repetir un bloque de código. | while (condición) { // código } | Mientras (condicion) se cumpla, ejecutar código. |
| **Do-While** | Como el While, pero el **bloque se ejecutará al menos una vez**. | do { // código } while (condición); | Hacer código, mientras (condicion) se cumpla. |
| **For** | Cuando **sabemos cuántas veces se repetirá el bloque y usamos un contador**. | for (inicialización; condición; incremento) { // código } | Para (inicialización; condición; incremento), ejecutar código. |

## Consideraciones:

- En los **bucles**, es importante **asegurarse de que la condición eventualmente se vuelva falsa para evitar bucles infinitos**.
- En los **bucles la variable de control (contador) debe ser declarada antes del bucle** si se va a utilizar después del mismo.
- Si queremos **imprimir un valor que no se calcula hasta que termine el bucle**, hacemos la **impresión fuera del bucle**.
- Si queremos imprimir un valor que se **calcula en cada iteración del bucle, hacemos la impresión dentro del bucle**.
- Los **operadores de incremento y decremento** son:
  - `++` es el operador de incremento que aumenta el valor de una variable en 1. Puede usarse como prefijo (`++i`) o sufijo (`i++`).
  - `--` es el operador de decremento que disminuye el valor de una variable en 1. Puede usarse como prefijo (`--i`) o sufijo (`i--`).
- Operadores de **asignación compuesta**:
  - `+=` y `-=` Permiten incrementar o decrementar una variable en una cantidad específica. Por ejemplo, `i += 2` es equivalente a `i = i + 2`.
  - `*=` y `/=` Permiten multiplicar o dividir una variable en una cantidad específica. Por ejemplo, `i *= 3` es equivalente a `i = i * 3`.